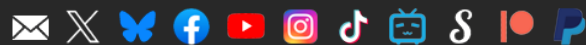


Inigo Quilez :: articles :: snap45 - 2025

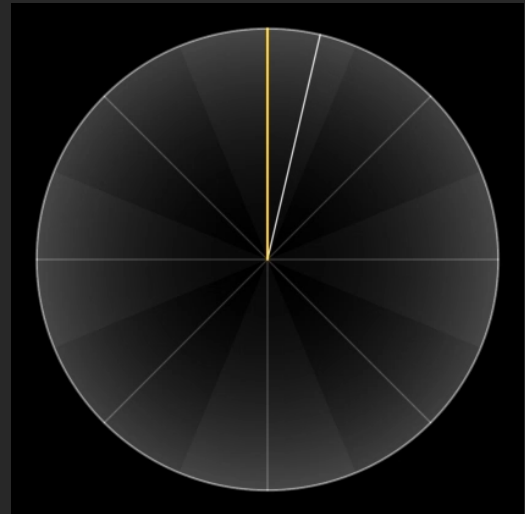


Intro

This is another example of avoiding trigonometry that came up the other day. I recommend reading [Part I](#) and [Part II](#) of this miniseries where I propose that using trigonometric functions (sin, cos, tan) or their inverses (asin, acos, atan) in any geometry code or rendering system is always a code smell.

This time, we'll look at a challenge that got sent my way when I claimed one can always avoid the rather expensive atan()/atan2() function. The problem is that of snapping a 2D unit vector to the closest 45 degree angle (N, NE, E, SE, S, SW, W, NW). The claim was that you do need to commute the angle of such a vector, round it to the closest 45 degree angle, then reconstruct the vector. Something like this perhaps:

```
vec2 snap45( in vec2 v )
{
    const float r = 6.283185/8.0; // 45 degree
    float a = atan(v.y,v.x);      // vector to
    float s = r*round(a/r);       // snap angl
    return vec2(cos(s),sin(s));   // angle to
}
```



This seems reasonable, and it does work - on the right side you can see how as the vector v in white moves, the yellow vector snaps to the closest 45 degrees direction. The triangular sectors are the basins of attraction of each direction. All good. But my claim was that surely you wouldn't need trigonometry to achieve the same behavior, and that if this code happened to be in a perf critical path, you'd want to do better.

Basic proposal

My immediate proposal was the most obvious one: all quadrants of the plane require the same treatment, so first use symmetry by taking the absolute value of the vector. Then realize that within the basic quadrant, there are only three possible outputs: horizontal (0 degree), diagonal (45 degree) and vertical (90 degree). The three cases can be easily determined by whether the x component of the vector is below 45-45/2 degrees or above 45+45/2 degrees. So the code I answered with was:

```
vec2 snap45( in vec2 v ) // 1.8x faster
{
    vec2 s = sign(v);
    float x = abs(v.x);
    return x>0.923880?vec2(s.x,0.0):
           x>0.382683?s*sqrt(0.5):
           vec2(0.0,s.y);
}
```

Here **0.923880** is $\cos(\pi/8)$, that is, the **x** component of a vector at 45-45/2 degrees, and **0.382683** is $\cos(3\pi/8)$, that is, the **x** component of a vector at 45+45/2 degrees.

I wouldn't consider this logic and corresponding code an "optimized" one, just a common sense approach to not doing the stupid thing to begin with. Yet, it already performs 1.8 times faster than the naive `atan()` based approach.

A better approach

Once I responded with this code I considered the challenge over, and so I archived the little shader code in Shadertoy for others to reuse. But as it often happens with the Shadertoy community, it didn't take long for somebody to take my common sense approach and make it look bad. The user Spalmer, helped by a correction provided by user FordPerfect, came up with this equivalent code:

```
vec2 snap45( in vec2 v ) // 2.1x faster
{
    v = round(v*1.30656296); // sqrt(1.0+sqrt(0.5)) = sin(π/8)/2
    return normalize(v);
}
```

This code uses a different approach, but before we explored, this code runs 2.1 times faster than the original, ie, slightly faster than mine. The way it works is that instead of thinking of snapping in angular space, you can think of a 2D grid covering the plane, such that we'll snap the vector to this grid. The value 1.30656296 is chosen such that the grid intersects exactly the 45-45/2 and 45+45/2 points; that means it is $\frac{1}{2}\sin(\pi/8) =$

$\sqrt{(1+\sqrt{1/2})}$. If this seems a bit abstract at first, a drawing will convince you this is what you need.

While this code was more than twice faster than the original `atan()` based one, I wasn't satisfied because it needs a dot product and a inverse square root for the `normalize()` call. So I made my own contribution to it and further optimized it to this:

```
vec2 snap45( in vec2 v ) // 2.8x faster
{
    v = round(v*1.30656296); // sqrt(1.0+sqrt(0.5)) = sin(π/8)/2
    return v*(abs(v.x)+abs(v.y)>1.5?sqrt(0.5):1.0);
}
```

This now runs **2.8 times faster** than the original `atan()` based implementation, not bad! You can see the source code for all these variants of the code here: <https://www.shadertoy.com/view/M3ycWd>.

Conclusion

So, as I claimed **more than a decade ago**, you can solve almost all geometry or rendering problems without trigonometry, in a more intuitive and efficient way (and generalizable, if you care about that)

But a more important takeaway is perhaps that if you move past the "general solution" and decide to specialize, you always discover lots of structure in the problem that can be exploited to simplify code (more readable) and optimize. In this example, the uncovered structure is the mirroring symmetry.

Programmers often forget this, or avoid it on purpose sometimes because of laziness, sometimes because they love to generalize and abstract (they are trained to always do so after all). But this is in many cases a mistake - specializing makes code simpler (and often more readable) and faster, and should be practiced in conjunction. Knowing when to abstract and when not to is an art. But try to always think when it makes sense to do so. This is by the way true in almost all contexts I can think of - solving Newtonian $F=ma$ is more efficient than solving relativistic motion, and easier to grasp. The Unreal Engine 5 can't beat a custom engine designed for one particular game, which will also have simpler codebase to navigate. And `"lea eax, [eax*4+eax]"` outperforms `"imul eax,5"`, although in that case the optimization has obscured the intent, which can be solved with a comment or a properly named function around it.

inigo quilez - learning computer graphics since 1994